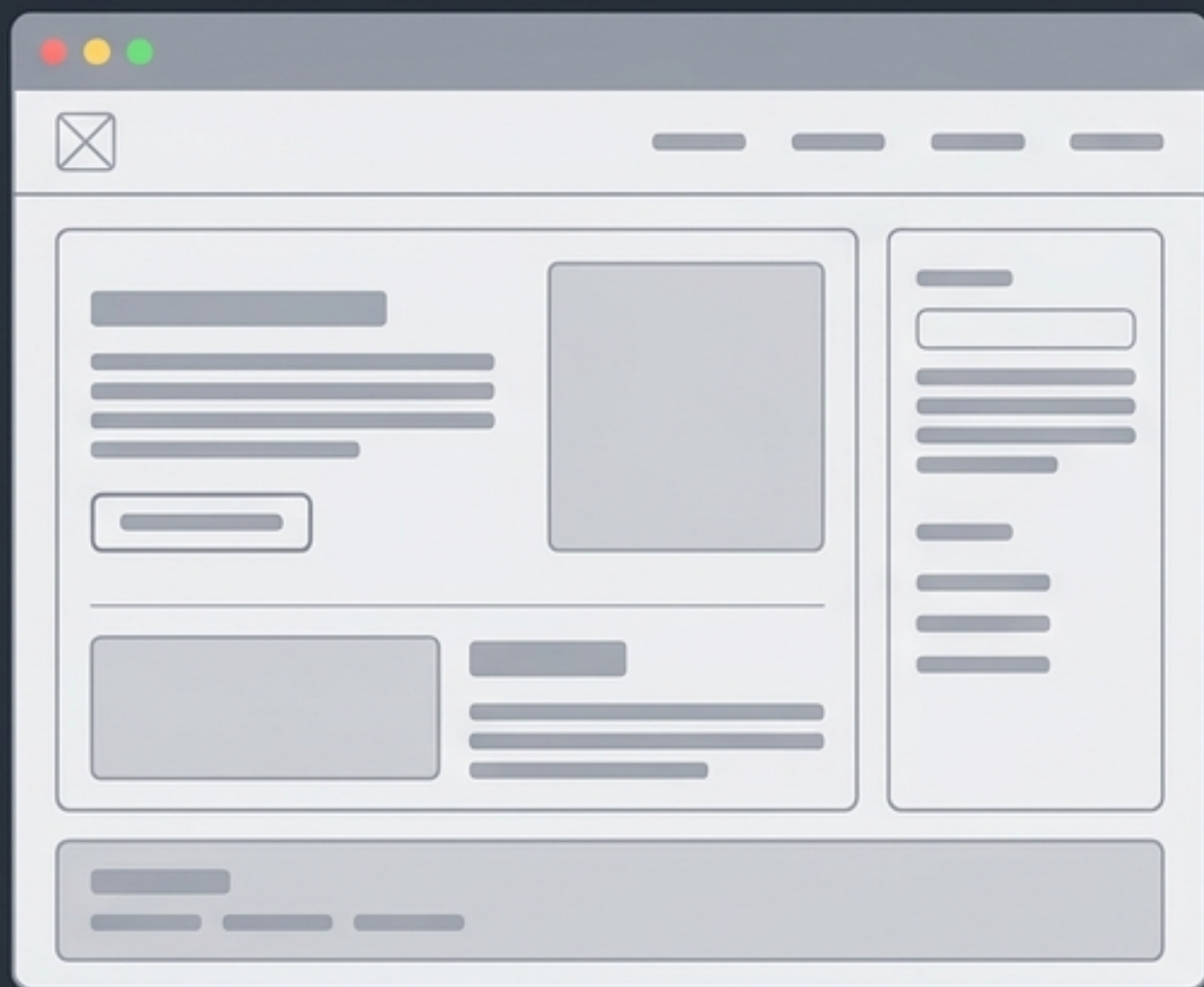




JavaScript: Los Bloques de Construcción de la Interactividad

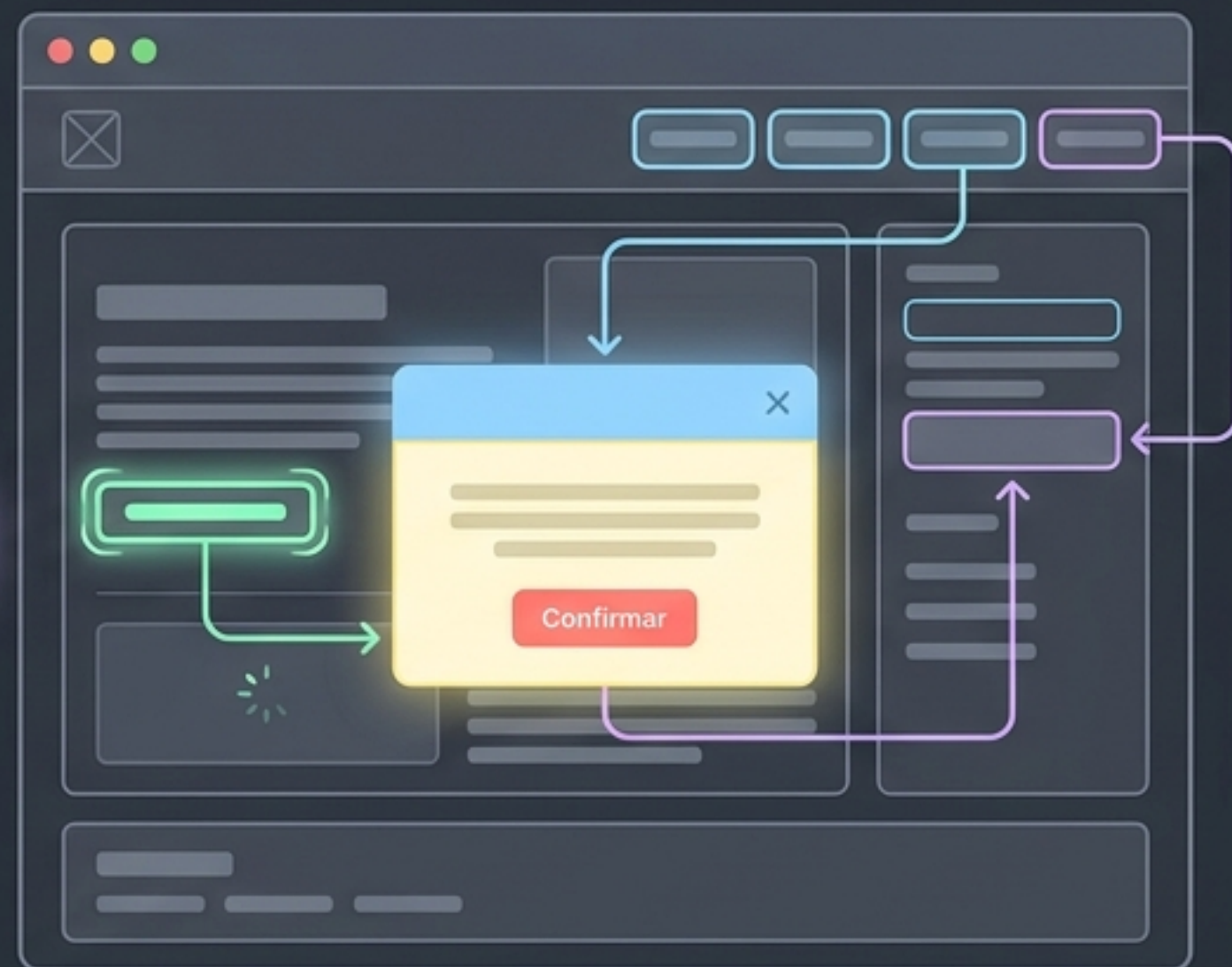
Fundamentos, lógica y sintaxis para dar vida a la web.

HTML/CSS (Estático)

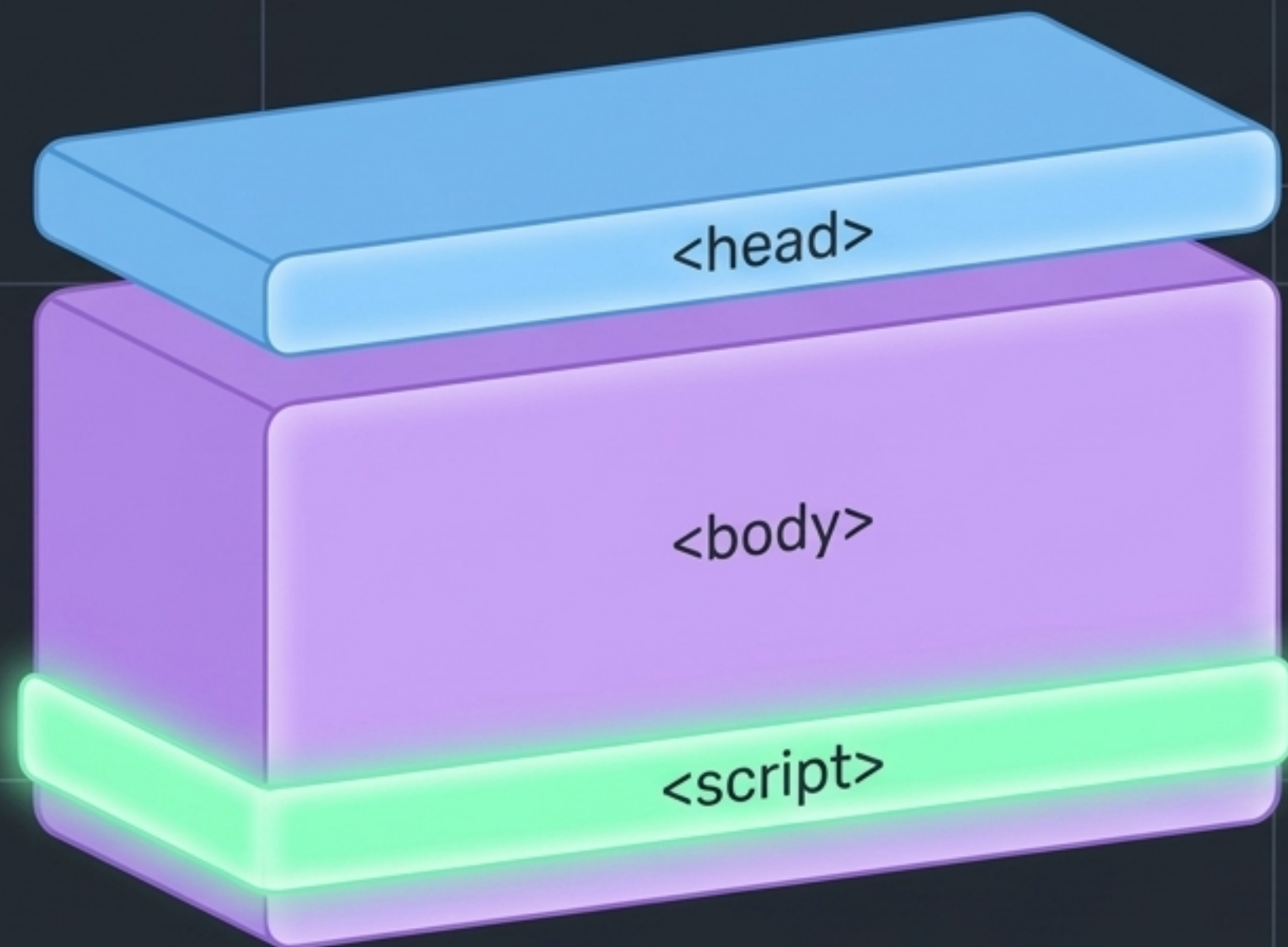


El Lenguaje
de Alto Nivel

JavaScript (Interactivo)



- Ejecutado directamente en el navegador del usuario.
- Transforma páginas estáticas en experiencias en tiempo real sin recargar (sin pestañeos).
- Alto Nivel: Diseñado con abstracción para ser legible por humanos, alejándose del código de máquina.



Buena Práctica: Colocar la etiqueta `<script>` al final, justo antes del cierre de `</body>`, permitiendo que el navegador cargue el contenido HTML antes de ejecutar las instrucciones.



Incrustado Directamente

Código colocado directamente entre etiquetas.

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Mi Página Web</title>
  </head>
  <body>
    <p>Contenido de la página.</p>
    <script>
      // Tu código de javascript aquí
      alert("¡Hola mundo!");
    </script>
  </body>
</html>
```



Enlazado Externamente

Vincula un archivo .js independiente mediante el atributo src.

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Mi Página Web</title>
  </head>
  <body>
    <p>Contenido de la página.</p>
    <script src="mi-script.js"></script>
  </body>
</html>
```


Comentarios (Para Humanos)

```
// Comentario de una línea: Explicación breve

/*
  Comentario de múltiples líneas:
  Explicación extensa que no impacta
  el comportamiento del programa.
*/
```

Consola (Para Desarrolladores)

 `.log()` → Mensaje estándar

 `.warn()` → Advertencia (Ícono amarillo)

 `.error()` → Error (Ícono rojo)

 `.info()` → Informativo

```
let nombre = "Juan";
let edad = 30;

console.log("nombre:", nombre);
console.log("Edad:", edad);
```



```
alert("Mensaje de la alerta");
```

La función que invoca la ventana emergente.

El argumento de texto, encerrado en comillas.

Mensaje de la alerta

OK

Casos de Uso



Notificar: Informar sobre situaciones importantes.



Confirmar: Prevenir acciones accidentales.

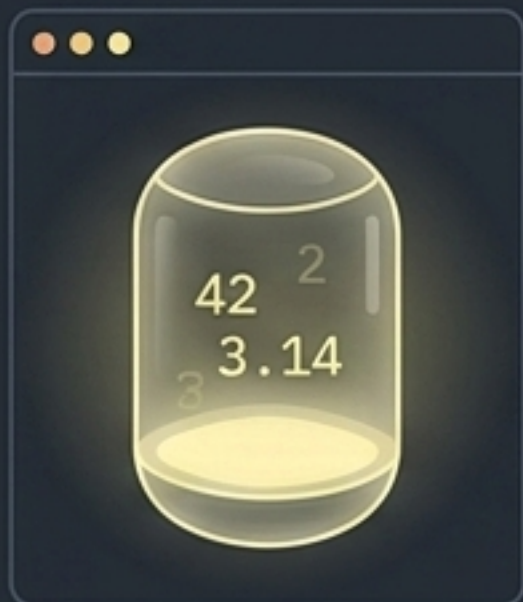


Advertir: Mostrar mensajes de error de entrada.



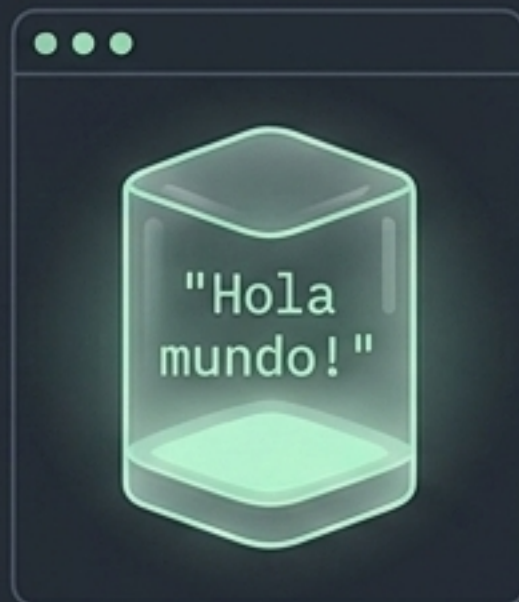
Tipado Dinámico

Las variables son contenedores que pueden cambiar de tipo durante la ejecución del programa.



Number

Valores numéricos (enteros o decimales).



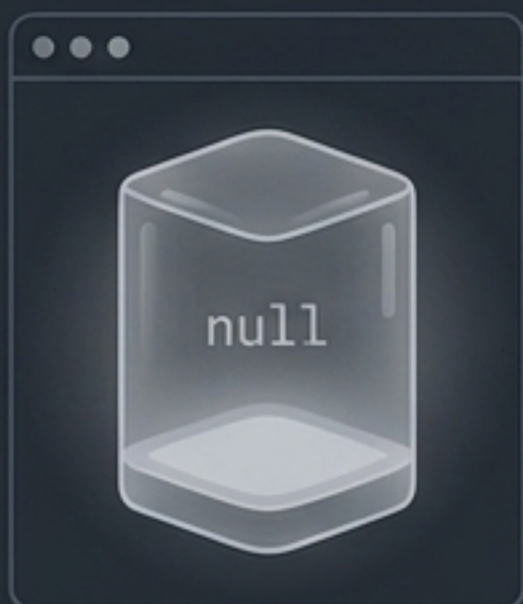
String

Secuencias de caracteres (encerradas en comillas).



Boolean

Verdadero (true) o falso (false).



null

Ausencia intencional de un valor.



Object

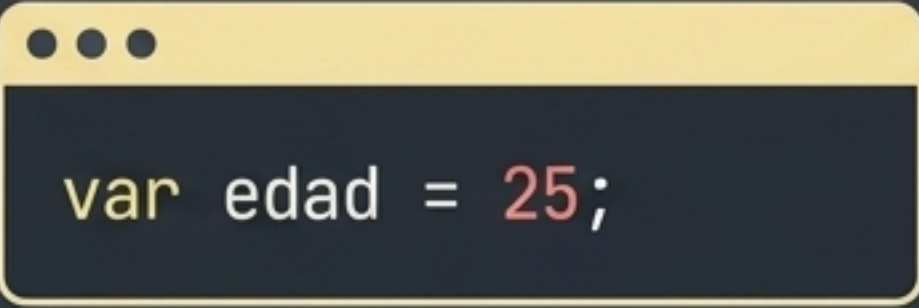
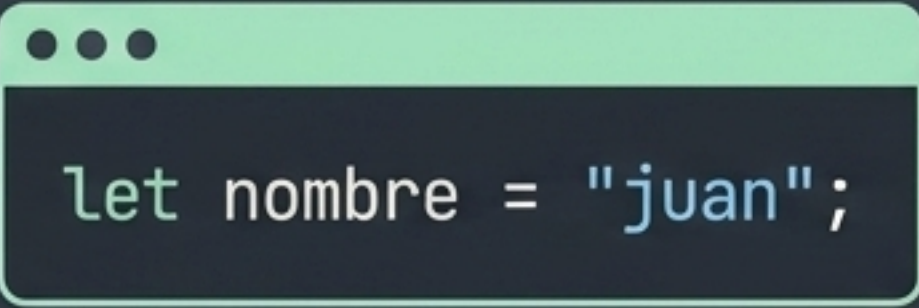
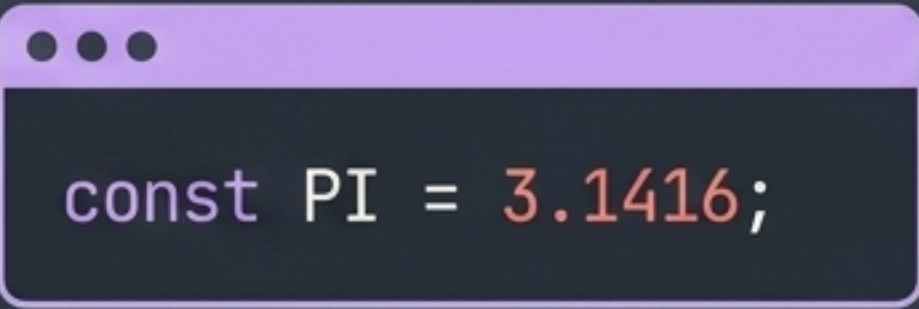
Colecciones de propiedades.



Array

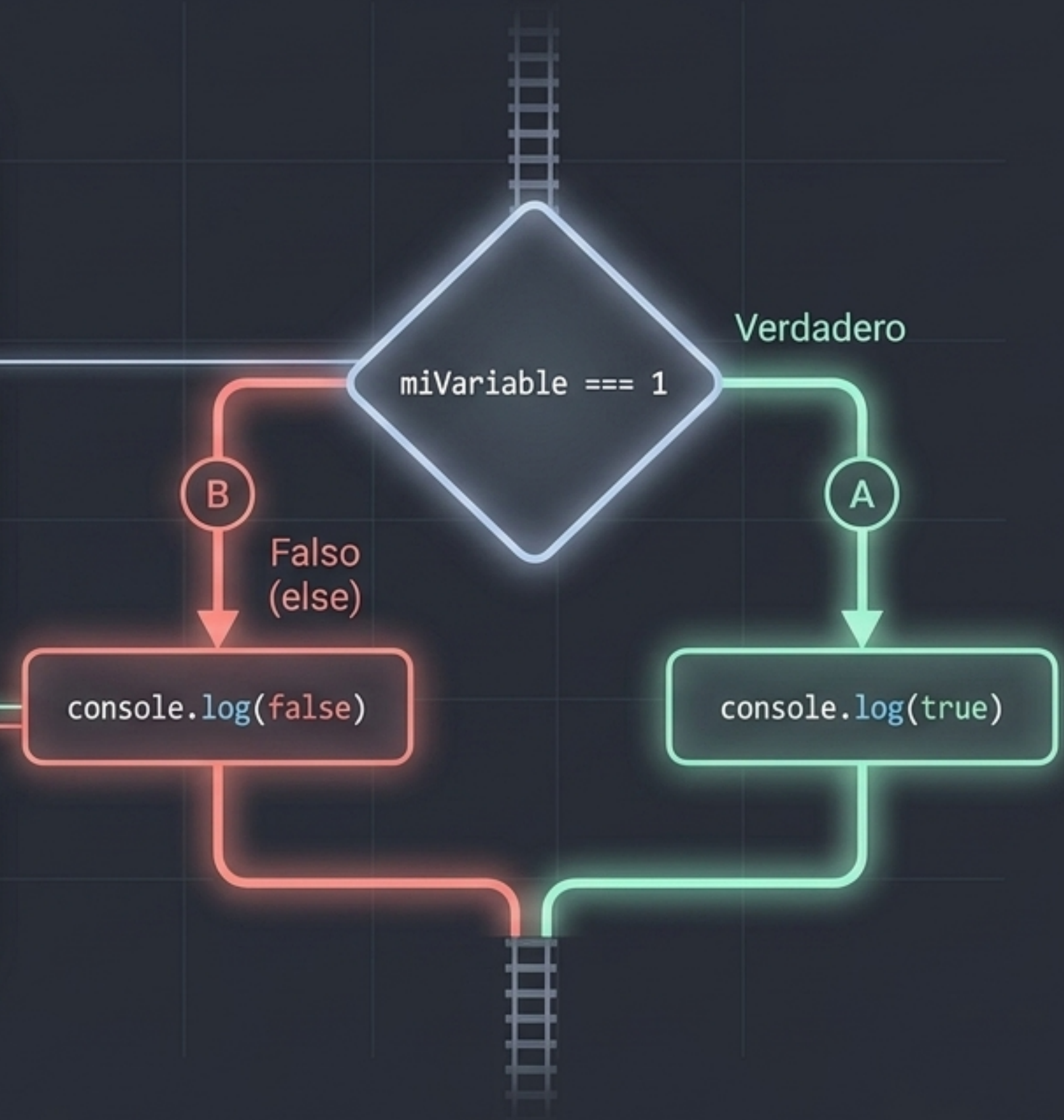
Listas de valores ordenados.

Matriz Evolutiva de Variables

Declaración	Ámbito (Scope)	Mutabilidad	Uso Recomendado
 <pre>var edad = 25;</pre>	Función	Sí	No recomendado en código moderno.
 <pre>let nombre = "juan";</pre>	Bloque {}	Sí (reassignable)	Estándar actual para valores que cambian.
 <pre>const PI = 3.1416;</pre>	Bloque {}	No (inmutable)	Para valores fijos y constantes.

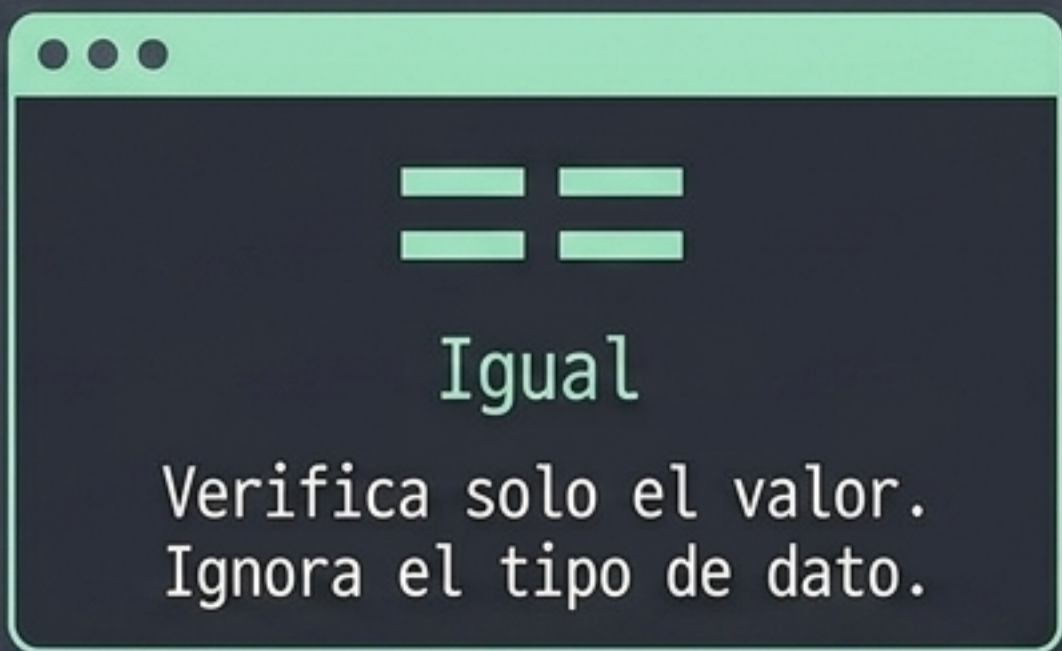
Operadores Condicionales: Tomando Decisiones

```
// Supongamos que tienes una variable llamada "miVariable"
var miVariable = 1; // Puedes cambiar el valor a 0 o 1 para probar
if (miVariable === 1) {
    // Cuando la variable es igual a 1, se retornará "verdadero".
    console.log(true);
} else {
    // En todos los demás casos (cuando la variable es 0 o
    // cualquier otro valor),
    // se retornará "falso".
    console.log(false);
}
```



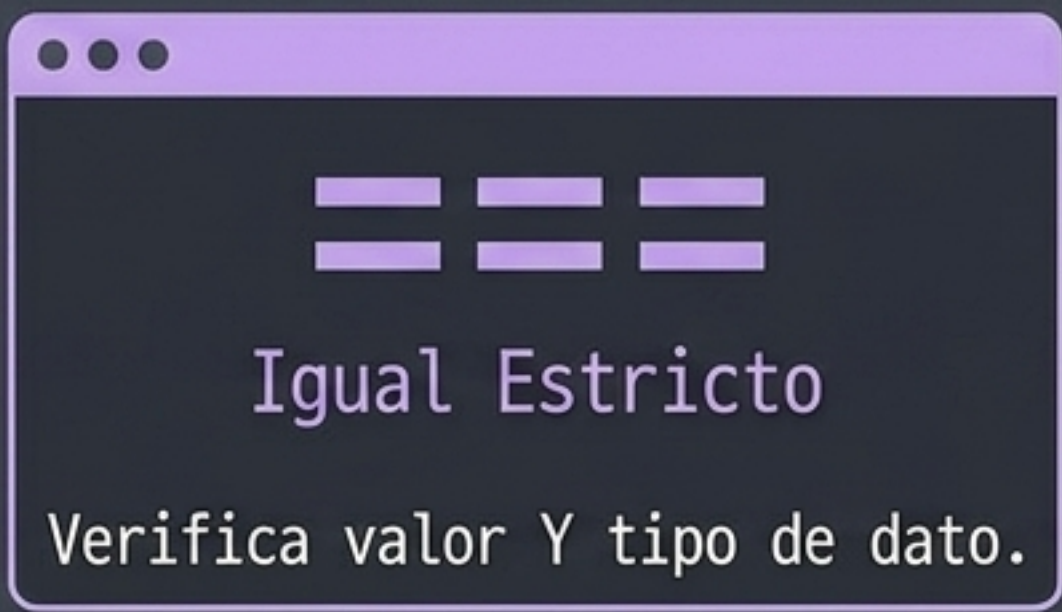
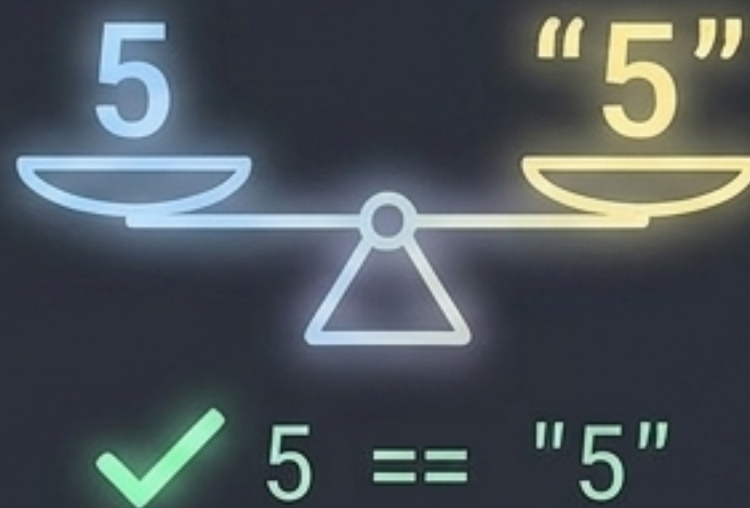
La Diferencia Crítica

Otros Operadores



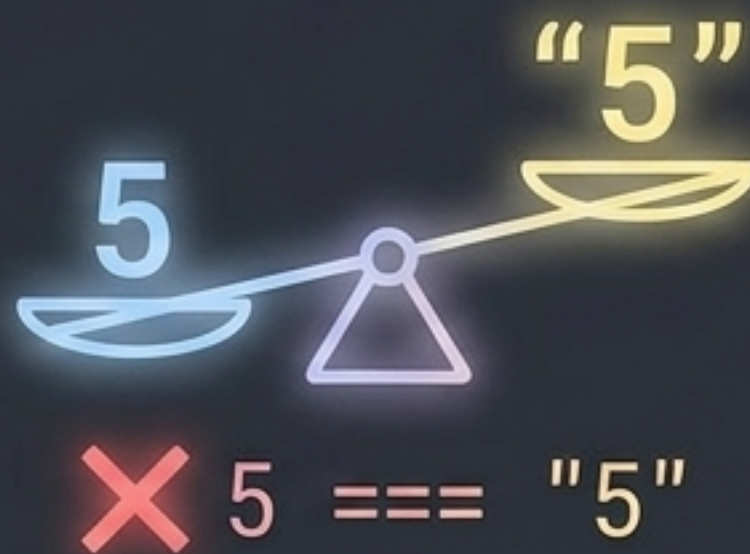
Igual

Verifica solo el valor.
Ignora el tipo de dato.



Igual Estricto

Verifica valor Y tipo de dato.



$!=$

Distinto

$!==$

Distinto
estricto

$>$

Mayor que

$<$

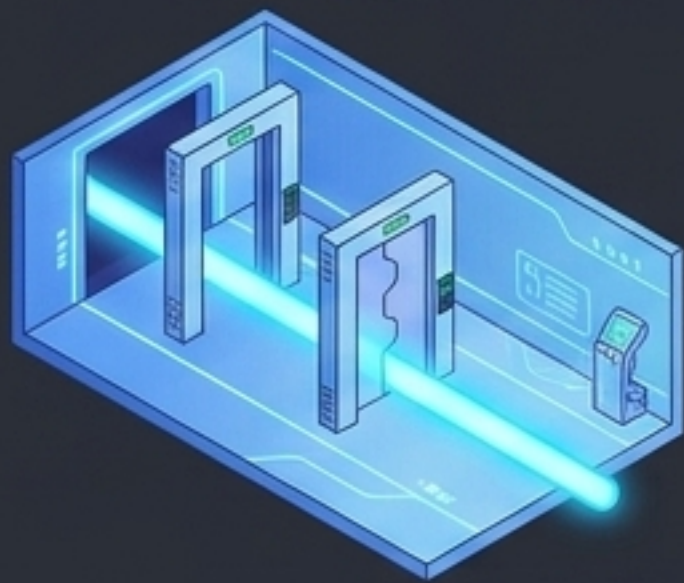
Menor que

$>=$

Mayor o igual

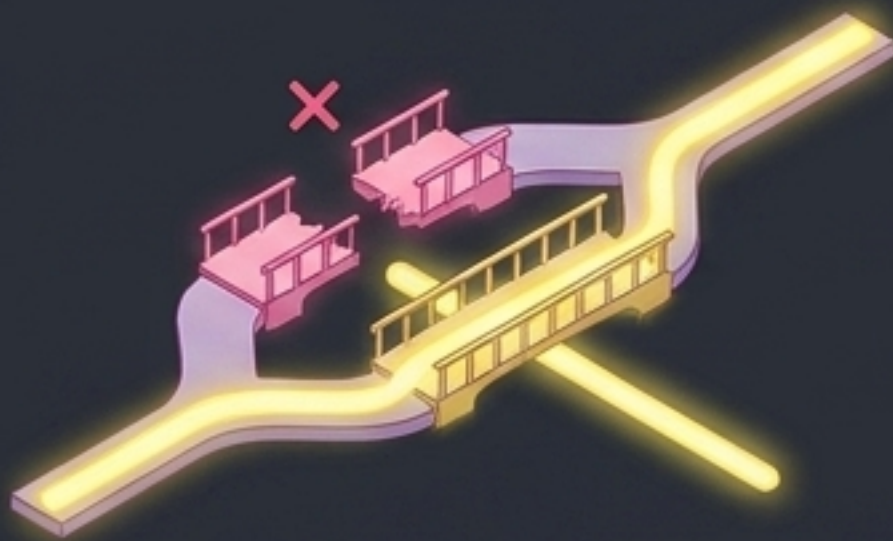
$<=$

Menor o igual



AND (&&) – El Pasillo de Doble Puerta

```
if (condicion1 && condicion2) {  
    // Esta condición se cumple si ambas condicion1 y condicion2 son verdaderas.  
}
```



OR (||) – Los Caminos Paralelos

```
if (condicion1 || condicion2) {  
    // Esta condición se cumple si al menos una de las condiciones, condicion1 o condicion2, es verdadera.  
}
```



NOT (!) – El Espejo Inversor

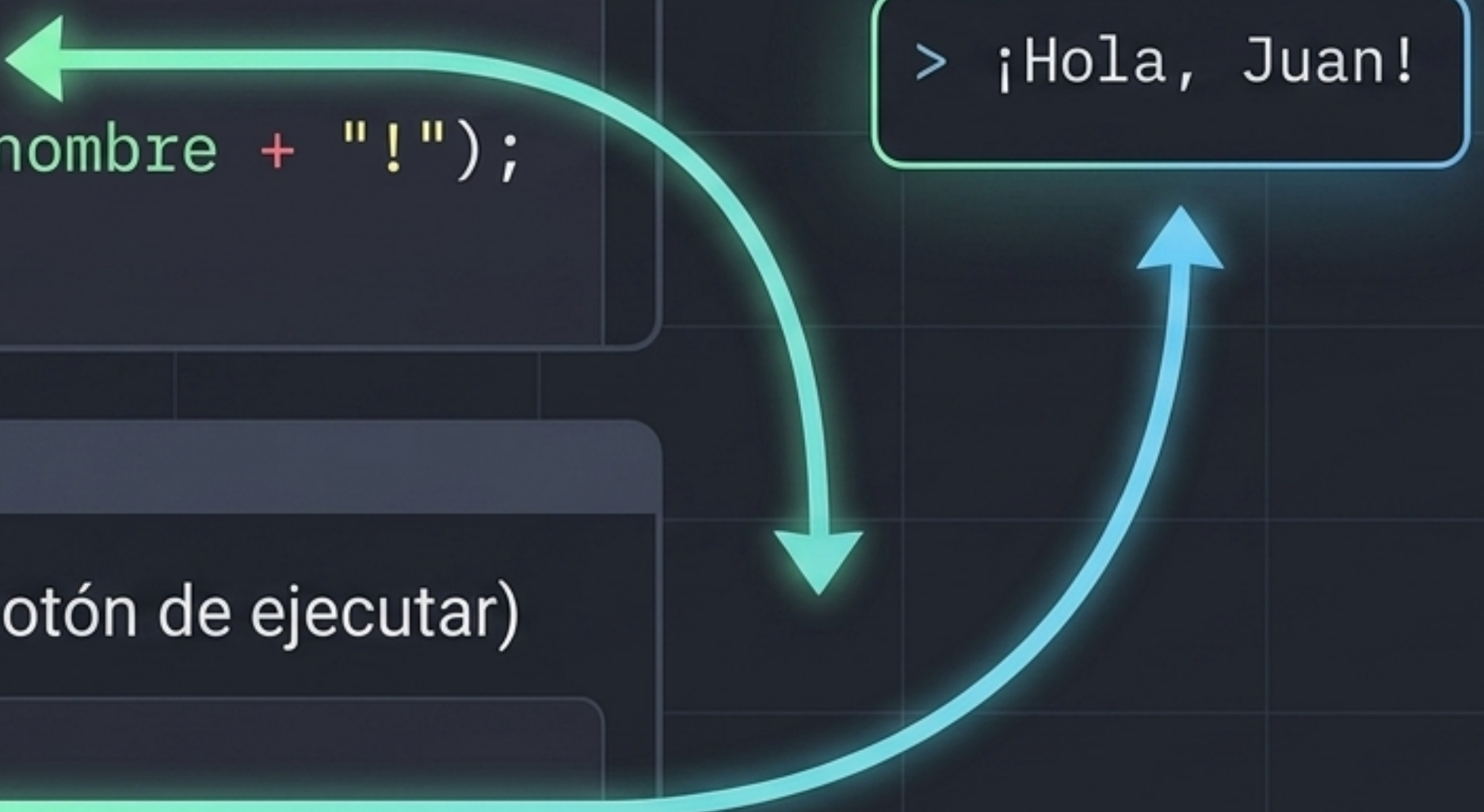
```
if (!condicion) {  
    // Esta condición se cumple si la condición original es falsa.  
}
```


Funciones: Empaquetando la Lógica



Definición (Guarda las instrucciones)

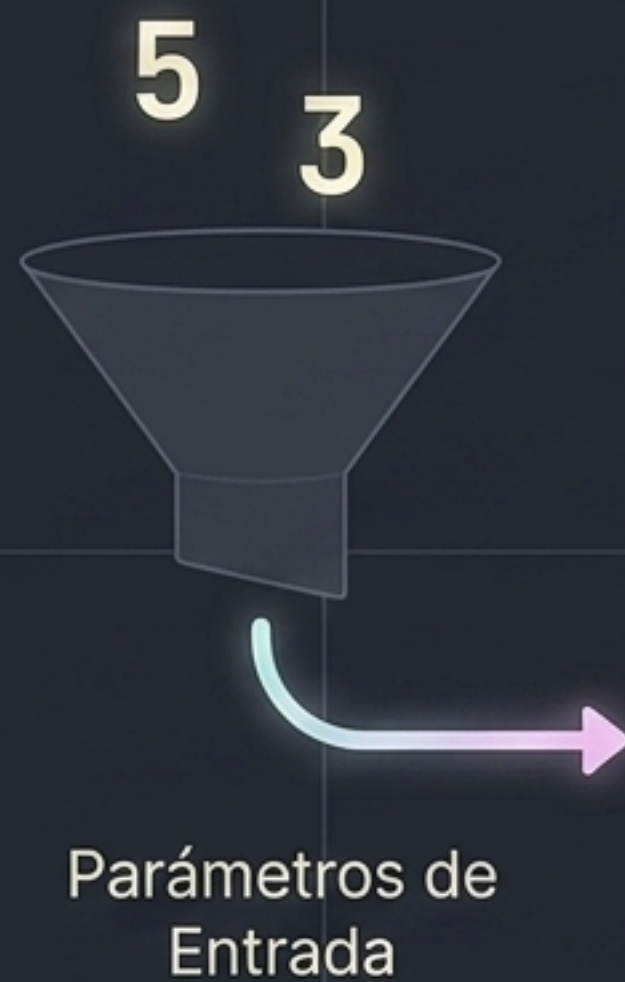
```
function saludar(nombre) {  
  console.log("¡Hola, " + nombre + "!");  
}
```



> ¡Hola, Juan!

Llamada / Invocación (Apretar el botón de ejecutar)

```
saludar("Juan");
```

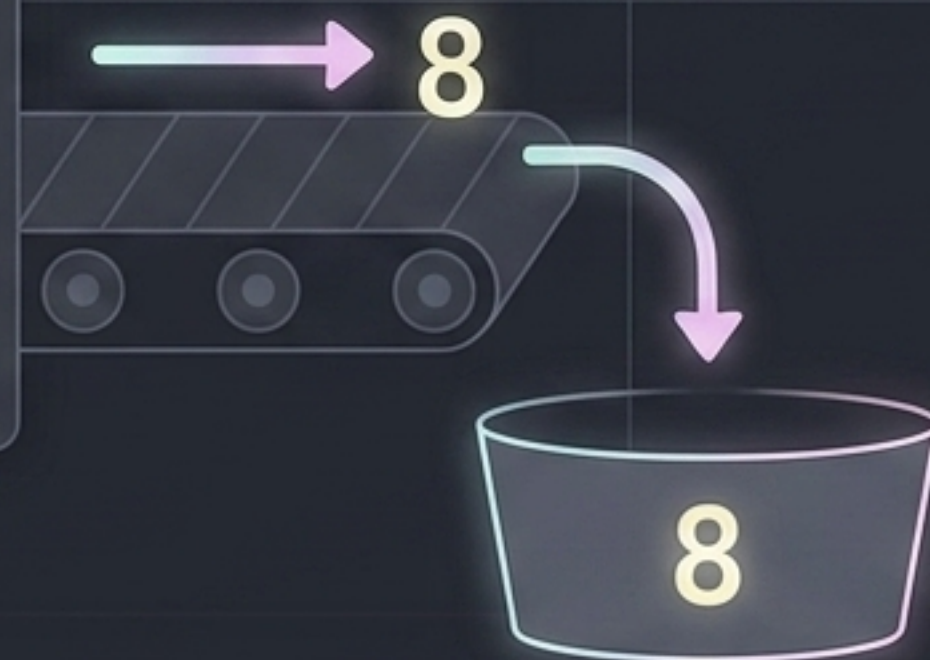



```
suma(a, b)

function suma(a, b) {
  let resultado = a + b;
  return resultado;
}
```

return detiene la ejecución interna y escupe el valor de vuelta al lugar donde fue llamada la función.

return



```
let resultadoSuma = suma(5, 3);
```


Variables:

Almacenan los datos de la realidad.

Lógicos: Combinan múltiples verdades.

```
function verificarAcceso(edad, tienePase) {  
  const edadMinima = 18;  
  if (edad >= edadMinima && tienePase === true) {  
    return "Acceso Permitido";  
  } else {  
    return "Acceso Denegado";  
  }  
}  
  
let resultado = verificarAcceso(20, true);  
console.log(resultado);
```

Relacionales:

Evalúan la verdad de los datos.

Funciones:

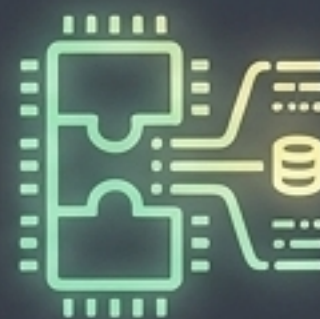
Empaquetan la lógica para reutilizarla.

Tu Entorno de Trabajo Está Listo



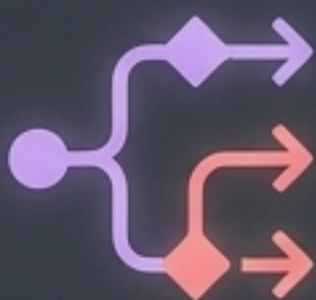
Consola & Alertas

Comunicación directa con el sistema y el usuario.



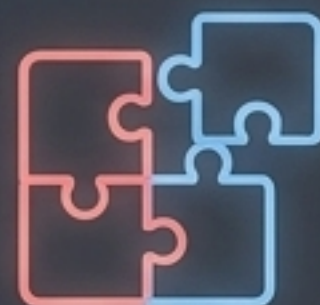
Variables (let/const)

Gestión segura y moderna de los datos.



Árboles de Decisión

Inteligencia y ruteo del flujo del programa mediante if/else y operadores.



Funciones

Modularidad, limpieza y reutilización del código.

Estos son los **cimientos lógicos** absolutos. Con estos bloques, estás preparado para automatizar tareas, manipular datos y, eventualmente, **dominar** frameworks modernos.